2

操作系统简介

如果你正在上本科的操作系统课，你应该已经大致知道计算机程序运行时在做什么了。如果不知道，这本书（以及对应的课程）会很难学。所以你也许该先放下这本书，或者赶紧跑去最近的书店，把必要的背景知识补上再回来（Patt 和 Patel 的书 [PP03]、Bryant 和 O'Hallaron 的书 [BOH10] 都相当不错）。

那么，程序运行时会发生什么？

嗯，运行中的程序只做一件非常简单的事：执行指令。每秒钟有数百万次（如今甚至是数十亿次），处理器从内存中**取**（fetch）出一条指令，对它**译码**（decode）（也就是弄清楚这是哪条指令），然后**执行**（execute）它（也就是做这条指令该做的事，比如把两个数相加、访问内存、检查某个条件、跳转到某个函数，等等）。执行完这条指令后，处理器接着处理下一条指令，如此往复，直到程序最终结束[1]。

这就是我们刚刚描述的**冯·诺依曼**（Von Neumann）计算模型的基本内容[2]。听起来很简单，对吧？但在这门课里我们会了解到，程序运行时还有许多别的事情在热火朝天地进行着，主要目的就是让系统**易于使用**（easy to use）。

事实上，有这样一组软件，它负责让程序运行起来变得容易（甚至让你看起来能同时运行很多程序），让程序共享内存，让程序能和设备交互，还有其他诸如此类的有趣事情。这组软件叫作**操作系统**（operating system，**OS**）[3]，因为它负责确保系统以易于使用的方式正确、高效地运转。

[1]当然，现代处理器为了让程序跑得更快，在底层做了许多稀奇古怪、让人心惊的事情，比如同时执行多条指令，甚至乱序发出和完成指令！不过这些不是我们这里关心的；我们只关心大多数程序所假定的那个简单模型：指令看上去是一条接一条、井然有序地依次执行的。

[2]冯·诺依曼是计算机系统的早期先驱之一。他在博弈论和原子弹方面也做过开创性的工作，还在 NBA 打过六年球。好吧，这几件事里有一件不是真的。

[3]操作系统早期还有一个名字叫**监督程序**（supervisor），甚至叫**主控程序**（master control program）。显然，后一个名字听起来有点过于野心勃勃（详见电影《Tron》），所以谢天谢地，最后流行开来的是“操作系统”这个名字。

关键问题：
如何虚拟化资源

本书要回答的一个核心问题非常简单：操作系统如何虚拟化资源？这就是我们问题的关键。操作系统为什么要这样做不是主要问题，答案应该显而易见：这让系统更容易使用。因此，我们关注的是怎么做：操作系统实现了哪些机制和策略来做到虚拟化？操作系统怎样高效地做到这一点？需要哪些硬件支持？

我们会用像这样带底色的“关键问题”框，点出构建操作系统时要解决的具体问题。因此，在关于某个主题的讲义中，你可能会看到一个或多个关键问题（cruces，没错，这才是它正确的复数形式）来突出要解决的问题。当然，章节里的细节会给出解决方案，或者至少给出解决方案的基本要素。

操作系统做到这一点的主要途径，是一种我们称为**虚拟化**（virtualization）的通用技术。也就是说，操作系统拿来一个**物理**（physical）资源（比如处理器、内存或者磁盘），把它变成一个更通用、更强大、更易用的**虚拟**（virtual）形式。因此，我们有时把操作系统称为**虚拟机**（virtual machine）。

当然，为了让用户能告诉操作系统做什么，从而使用虚拟机的各种功能（比如运行程序、分配内存或访问文件），操作系统还提供了一些可供调用的接口（API）。事实上，一个典型的操作系统会向应用程序提供几百个**系统调用**（system call）。由于操作系统提供这些调用来运行程序、访问内存和设备，以及做其他相关的事，我们有时也说操作系统为应用程序提供了一个**标准库**（standard library）。

最后，由于虚拟化让许多程序得以运行（从而共享 CPU），让许多程序得以并发地访问各自的指令和数据（从而共享内存），让许多程序得以访问设备（从而共享磁盘等），操作系统有时也被称为**资源管理器**（resource manager）。CPU、内存和磁盘都是系统的**资源**（resource）；因此操作系统的职责就是**管理**（manage）这些资源，并且要做得高效、公平，或者兼顾其他许多可能的目标。为了更好地理解操作系统的职责，我们来看几个例子。

2.1 虚拟化 CPU

图 2.1 是我们的第一个程序。它做的事不多。实际上，它只是调用 `Spin()`，这个函数反复检查时间，运行满一秒后就返回。然后，程序打印出用户在命令行中传入的字符串，并永远这样重复下去。

假设我们把这个文件保存为 `cpu.c`，然后在只有一个处理器（我们有时也叫它 **CPU**）的系统上编译并运行它。我们会看到这样的结果：

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <assert.h>
#include "common.h"

int
main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: cpu <string>\n");
        exit(1);
    }
    char *str = argv[1];
    while (1) {
        Spin(1);
        printf("%s\n", str);
    }
    return 0;
}
```

图 2.1：**简单的例子：循环打印的代码（`cpu.c`）**

```
prompt> gcc -o cpu cpu.c -Wall
prompt> ./cpu "A"
A
A
A
A
^C
prompt>
```

这次运行没什么意思：系统开始运行程序，程序反复检查时间，直到过了一秒。一秒过后，代码打印出用户传入的字符串（在这个例子中是字母“A”），然后继续。注意，这个程序会永远运行下去；按下“Control-c”（在基于 UNIX 的系统上，这会终止在前台运行的程序）就能让它停下来。

现在我们做同样的事，但这次运行同一个程序的多个实例。图 2.2 展示了这个稍微复杂一点的例子的结果。

嗯，现在事情变得有意思一点了。尽管只有一个处理器，这四个程序却似乎都在同时运行！这个魔法是怎么变出来的[4]?

原来，是操作系统在硬件的一些帮助下，营造了这个**假象**（illusion），也就是系统拥有大量虚拟 CPU 的假象。把一个 CPU（或者少数几个 CPU）变成看似无限多个 CPU，从而

[4]注意，我们是用 & 符号同时运行四个进程的。在 zsh shell 中，这样做会把作业放到后台运行，于是用户可以马上发出下一条命令，在这个例子中就是再运行一个程序。如果你用的是别的 shell（例如 tcsh），做法会略有不同；详情请查阅网上的文档。

```
prompt> ./cpu A & ./cpu B & ./cpu C & ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
...
```

图 2.2：**同时运行多个程序**

让许多程序看起来同时在运行，这就是我们所说的**虚拟化 CPU**（virtualizing the CPU），它是本书第一大部分的重点。

当然，要运行程序、停止程序，或者以其他方式告诉操作系统运行哪些程序，就需要一些接口（API），让你把自己的意图传达给操作系统。整本书我们都会谈到这些 API；事实上，它们是大多数用户与操作系统打交道的主要方式。

你也许还会注意到，能同时运行多个程序，带来了各种新问题。例如，如果两个程序都想在某个时刻运行，应该让哪个运行？这个问题由操作系统的**策略**（policy）来回答；操作系统中许多不同的地方都用策略来回答这类问题，所以我们在学习操作系统实现的基本**机制**（mechanism）（比如同时运行多个程序的能力）时，也会研究这些策略。这也正是操作系统作为**资源管理器**的职责所在。

2.2 虚拟化内存

现在来看内存。现代计算机提供的**物理内存**（physical memory）模型非常简单。内存就是一个字节数组；要**读**（read）内存，必须指定一个**地址**（address），才能访问存放在那里的数据；要**写**（write）（或**更新**，update）内存，还必须指定要写到这个地址的数据。

程序运行时，内存一直在被访问。程序把它的所有数据结构都放在内存中，并通过各种指令访问它们，比如加载和存储指令，或者其他在工作中显式访问内存的指令。别忘了，程序的每一条指令也都在内存中；因此，每次取指令都要访问内存。

我们来看一个通过调用 `malloc()` 分配内存的程序（见图 2.3）。这个程序的输出如下：

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include "common.h"

int
main(int argc, char *argv[])
{
    int *p = malloc(sizeof(int));                    // a1
    assert(p != NULL);
    printf("(%d) address pointed to by p: %p\n",
           getpid(), p);                             // a2
    *p = 0;                                          // a3
    while (1) {
        Spin(1);
        *p = *p + 1;
        printf("(%d) p: %d\n", getpid(), *p);        // a4
    }
    return 0;
}
```

图 2.3: **访问内存的程序（`mem.c`）**

```
prompt> ./mem
(2134) address pointed to by p: 0x200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

```
prompt> ./mem & ./mem &
[1] 24113
[2] 24114
(24113) address pointed to by p: 0x200000
(24114) address pointed to by p: 0x200000
(24113) p: 1
(24114) p: 1
(24114) p: 2
(24113) p: 2
(24113) p: 3
(24114) p: 3
(24113) p: 4
(24114) p: 4
...
```

图 2.4: **多次运行访问内存的程序**

这个程序做了几件事。首先，它分配了一些内存（a1 行）。然后，它打印出这块内存的地址（a2），再把数字零放进新分配内存的第一个位置（a3）。最后，它进入循环，每次延迟一秒，然后把 p 所保存的地址处的值加一。每条打印语句还会打印出运行中程序的所谓进程标识符（process identifier，PID）。每个运行中的进程都有唯一的 PID。

同样，第一次的结果没什么意思。新分配的内存位于地址 0x200000。程序运行时，它慢慢地更新这个值，并打印出结果。

现在，我们再次运行同一个程序的多个实例，看看会发生什么（图 2.4）。从这个例子可以看到，每个运行中的程序都在同一个地址（0x200000）分配了内存，但每个程序似乎都在独立地更新 0x200000 处的值！就好像每个运行中的程序都有自己的私有内存，而不是和其他运行中的程序共享同一块物理内存[5]。

事实上，这正是操作系统**虚拟化内存**（virtualizing memory）时发生的事情。每个进程访问的是它自己私有的**虚拟地址空间**（virtual address space，有时简称**地址空间**，address space），操作系统以某种方式把它映射到机器的物理内存上。一个运行中程序的内存引用，不会影响其他进程（或操作系统本身）的地址空间；就这个运行中的程序而言，它独享整个物理内存。然而现实是，物理内存是一种共享资源，由操作系统管理。这一切究竟是怎么做到的，也是本书第一部分（关于**虚拟化**的部分）要讨论的内容。

2.3 并发

本书的另一个主题是**并发**（concurrency）。我们用这个概念性的术语，指代在同一个程序中同时（即并发地）处理许多事情时出现、而且必须解决的一系列问题。并发问题最早出现在操作系统内部；从上面虚拟化的例子可以看到，操作系统同时应付着许多事情，先运行一个进程，再运行另一个，如此往复。事实证明，这样做会引出一些深刻而有趣的问题。

遗憾的是，并发问题已经不再局限于操作系统本身。事实上，现代**多线程**（multi-threaded）程序也表现出同样的问题。我们用一个**多线程**程序的例子来演示一下（图 2.5）。

虽然你现在可能还不能完全理解这个例子（后面并发部分的章节会讲很多相关内容），但基本思想很简单。主程序用 Pthread_create() 创建了两个**线程**（thread）[6]。你可以把线程看作和其他函数运行在同一个内存空间中的函数，而且同时有不止一个这样的函数处于活动状态。在这个例子中，每个线程都从一个叫 worker() 的例程开始运行，这个例程只是在循环中把一个计数器递增 loops 次。

下面是把变量 loops 的输入值设为 1000 时运行这个程序的记录。loops 的值决定了两个工作线程各自在循环中递增共享计数器多少次。当 loops 设为 1000 时，你预计 counter 的最终值是多少?

[5]要让这个例子成立，你需要确保关闭了地址空间随机化；事实证明，随机化可以很好地防御某些类型的安全漏洞。你可以自己去多了解一下，特别是如果你想学习如何通过栈溢出（stack-smashing）攻击闯入计算机系统的话。我们可不是在推荐这么做……

[6]实际应该调用的是小写的 pthread_create()；大写的版本是我们自己写的包装函数，它调用 pthread_create()，并确保返回码表明调用成功。详见代码。

```
#include <stdio.h>
#include <stdlib.h>
#include "common.h"
#include "common_threads.h"

volatile int counter = 0;
int loops;

void *worker(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        counter++;
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: threads <value>\n");
        exit(1);
    }
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);

    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value   : %d\n", counter);
    return 0;
}
```

图 2.5：**多线程程序（`threads.c`）**

```
prompt> gcc -o threads threads.c -Wall -pthread
prompt> ./threads 1000
Initial value : 0
Final value   : 2000
```

你可能已经猜到了，两个线程结束时，计数器的最终值是 2000，因为每个线程都把计数器递增了 1000 次。实际上，当 `loops` 的输入值设为 N 时，我们预计程序最终输出 $2N$。但事实证明，事情没这么简单。我们用更大的 `loops` 值运行同一个程序，看看会怎样：

```
prompt> ./threads 100000
Initial value : 0
Final value   : 143012  // 啊??
prompt> ./threads 100000
Initial value : 0
Final value   : 137298  // 这是什么鬼??
```

这次运行中，我们输入的值是 100,000，得到的最终值却不是 200,000，而是 143,012。然后，第二次运行这个程序时，我们不仅又得到了错误的值，而且这个值和上一次还不一样。事实上，如果你用很大的 `loops` 值反复运行这个程序，你可能会发现有时甚至能得到正确的答案！为什么会这样呢?

原来，这些奇怪而反常的结果，和指令的执行方式有关：指令是一次执行一条的。遗憾的是，上面程序中的关键部分，也就是递增共享计数器的地方，需要三条指令：一条把计数器的值从内存加载到寄存器，一条把它加一，一条把它存回内存。因为这三条指令不是**原子地**（atomically）执行的（即不是一下子全部执行完），所以会发生奇怪的事情。我们将在本书第二部分详细讨论的，正是这种**并发**问题。

关键问题：
如何构建正确的并发程序
当同一个内存空间中有许多并发执行的线程时，我们怎样才能构建一个正确工作的程序？需要操作系统提供哪些原语？硬件应该提供哪些机制？怎样用它们来解决并发问题?

2.4 持久性

这门课的第三个主题是**持久性**（persistence）。在系统内存中，数据很容易丢失，因为像 DRAM 这样的设备是以**易失**（volatile）的方式存储数据的；一旦断电或者系统崩溃，内存中的所有数据都会丢失。因此，我们需要能够**持久地**（persistently）存储数据的硬件和软件；这种存储对任何系统都至关重要，因为用户非常在乎自己的数据。

硬件方面是某种**输入/输出**（input/output，**I/O**）设备；在现代系统中，**硬盘驱动器**（hard drive）是存放长期信息的常用仓库，不过**固态硬盘**（solid-state drive，**SSD**）在这个领域也在不断进步。

操作系统中通常负责管理磁盘的那部分软件叫作**文件系统**（file system）；因此，它负责把用户创建的所有**文件**（file）可靠、高效地存放到系统的磁盘上。

和操作系统为 CPU 与内存提供的抽象不同，操作系统并不会为每个应用程序都创建一个私有的虚拟磁盘。相反，它假定用户常常想要**共享**（share）文件中的信息。例如，写 C 程序时，你可能先用编辑器（例如 Emacs[7]）创建并编辑 C 文件（`emacs -nw main.c`）。完成后，你可能会用编译器把源代码变成可执行文件（例如 `gcc -o main main.c`）。最

[7]你应该用 Emacs。如果你用的是 vi，那你大概有点什么问题。如果你用的根本不是一个真正的代码编辑器，那就更糟了。

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <assert.h>
4  #include <fcntl.h>
5  #include <sys/types.h>
6
7  int main(int argc, char *argv[]) {
8      int fd = open("/tmp/file",
9                    O_WRONLY|O_CREAT|O_TRUNC,
10                   S_IRWXU);
11     assert(fd > -1);
12     int rc = write(fd, "hello world\n", 12);
13     assert(rc == 12);
14     close(fd);
15     return 0;
16 }
```

图 2.6：**执行 I/O 的程序（`io.c`）**

后，你可能会运行这个新的可执行文件（例如 `./main`）。这样，你就能看到文件是怎样在不同进程之间共享的。首先，Emacs 创建了一个文件，作为编译器的输入；编译器用这个输入文件创建出一个新的可执行文件（中间要经过很多步骤，详情请去上编译原理课）；最后，这个新的可执行文件被运行起来。于是，一个新程序就诞生了！

为了更好地理解这一点，我们来看一些代码。图 2.6 中的代码创建了一个文件（`/tmp/file`），里面包含字符串“hello world”。

为了完成这项任务，程序向操作系统发出了三次调用。第一次调用 `open()`，打开并创建这个文件；第二次调用 `write()`，向文件写入一些数据；第三次调用 `close()`，只是关闭文件，表示程序不会再向它写入数据了。这些**系统调用**（system call）会被转到操作系统中叫作**文件系统**（file system）的那部分去，由它处理这些请求，并向用户返回某种错误码。

你可能想知道，操作系统为了真正写入磁盘都做了些什么。我们可以给你看，但你得先保证闭上眼睛；这个过程就是这么难看。文件系统要做不少工作：首先要弄清楚这些新数据放在磁盘上的什么位置，然后在文件系统维护的各种结构中记录下来。要做到这些，就需要向底层存储设备发出 I/O 请求，要么读取已有的结构，要么更新（写入）它们。写过**设备驱动程序**（device driver）[8]的人都知道，让设备替你做点事情，是一个复杂而琐碎的过程。它要求你对底层设备接口及其确切语义有深入的了解。幸运的是，操作系统通过系统调用，提供了一种标准而简单的访问设备的方式。因此，操作系统有时被看作一个**标准库**（standard library）。

当然，关于怎样访问设备、文件系统怎样在设备之上持久地管理数据，还有很多细节。出于性能考虑，大多数文件系统会先把这类写操作推迟一段时间，希望把它们合成更大的批次。为了应对写入过程中系统崩溃的问题，大多数文件系统都采用了某种复杂的写入协议，比如**日志**（journaling）或**写时复制**（copy-on-write），仔细安排写入磁盘的顺序，确保写

[8]设备驱动程序是操作系统中的一段代码，它知道如何与某个特定的设备打交道。后面我们会进一步讨论设备和设备驱动程序。

入过程中一旦出现故障，系统之后能够恢复到合理的状态。为了让各种常见操作都很高效，文件系统采用了许多不同的数据结构和访问方法，从简单的列表到复杂的 B 树。如果这些现在还讲不通，很好！我们会在本书关于**持久性**的第三部分详细讨论这些内容：先讨论一般的设备和 I/O，再详细讨论磁盘、RAID 和文件系统。

关键问题：
如何持久地存储数据
文件系统是操作系统中负责管理持久数据的部分。要正确地做到这一点，需要哪些技术？要获得高性能，需要哪些机制和策略？面对硬件和软件故障，怎样实现可靠性？

2.5 设计目标

现在你已经大致知道操作系统实际上做什么了：它拿来 CPU、内存或磁盘这样的物理**资源**，把它们**虚拟化**。它处理与**并发**有关的棘手问题。它还**持久地**存储文件，从而让文件长期保持安全。既然我们想构建这样一个系统，就需要心里有一些目标，帮助我们聚焦设计与实现，并在必要时做出权衡；找到恰当的一组权衡，是构建系统的关键。

最基本的目标之一，是建立一些**抽象**（abstraction），让系统方便、易用。抽象是我们在计算机科学中所做的一切的基础。有了抽象，我们才能把一个大程序拆成许多小而易懂的部分来编写；才能用 C[9] 这样的高级语言写程序，而不必考虑汇编；才能用汇编写代码，而不必考虑逻辑门；才能用逻辑门搭出处理器，而不必过多考虑晶体管。抽象是如此基本，以至于我们有时会忘了它的重要性；但在这里我们不会忘记。因此，在每一部分里，我们都会讨论一些随着时间发展起来的主要抽象，给你一种思考操作系统各个部分的方式。

设计和实现操作系统的一个目标是提供高**性能**（performance）；换一种说法，我们的目标是**尽量减少操作系统的开销**（minimize the overheads）。虚拟化和让系统易于使用是非常值得的，但不能不惜任何代价；因此，我们必须努力在没有过多开销的情况下提供虚拟化和其他操作系统功能。这些开销有多种形式：额外的时间（更多的指令）和额外的空间（内存中或磁盘上）。如果可能，我们会寻找能尽量减少其中一种或同时减少两种开销的解决方案。不过，完美并不总是能够达到的；我们要学会察觉这一点，并（在合适的时候）容忍它。

另一个目标是在应用程序之间、以及操作系统和应用程序之间提供**保护**（protection）。因为我们希望让许多程序同时运行，所以要确保其中一个程序的恶意或意外的不良行为不会伤害其他程序；我们当然更不希望某个应用程序能够伤害操作系统本身（因为那会影响系统上运行的所有程序）。保护是操作系统背后一条主要原则的核心，那就是**隔离**（isolation）；把进程彼此隔离开来是实现保护的关键，因此也是操作系统必须做的许多事情的基础。

操作系统还必须不间断地运行；一旦它出故障，系统上运行的所有应用程序也都会出故障。正因为有这种依赖，操作系统常常力求提供高度的**可靠性**（reliability）。随着操作系统变得越来越复杂（有时包含数百万行代码），构建一个可靠的操作系统是相当大的挑战；事

[9] 你们中有人可能会反对把 C 叫作高级语言。不过别忘了，这是一门操作系统课，在这里我们只要不用一直写汇编，就已经很开心了！

 © 2008–25, ARPACI-DUSSEAU

实上，这个领域正在进行的许多研究（包括我们自己的一些工作 [BS+09, SS+10]）关注的正是这个问题。

还有其他一些合理的目标：在我们这个越来越注重环保的世界里，**能效**（energy-efficiency）很重要；防范恶意应用程序的**安全性**（security）（其实是保护的延伸）至关重要，尤其是在如今这个高度联网的时代；随着操作系统运行在越来越小的设备上，**移动性**（mobility）也越来越重要。根据系统的使用方式，操作系统会有不同的目标，因此实现方式很可能至少会略有不同。不过我们会看到，我们将要介绍的许多构建操作系统的原则，在各种不同的设备上都很有用。

2.6 一点历史

在结束这篇简介之前，我们简要介绍一下操作系统的发展历史。和人类构建的任何系统一样，随着工程师们逐渐了解操作系统设计中什么才是重要的，好的想法也在操作系统中日积月累。这里我们讨论几项主要的发展。想了解更详细的内容，请看 Brinch Hansen 写的那篇出色的操作系统史 [BH00]。

早期操作系统：只是一些库

起初，操作系统做的事情不多。基本上，它只是一组常用函数的库；例如，与其让系统中的每个程序员都自己编写底层 I/O 处理代码，“操作系统”会提供这样的 API，从而让开发者的日子好过一些。

通常，在那些老式的大型机系统上，一次只运行一个程序，由一名人类操作员控制。你以为现代操作系统会做的许多事情（例如决定以什么顺序运行作业），当时都是由这名操作员来做的。如果你是个聪明的开发者，你就会对这位操作员好一点，这样他们也许会把你的作业挪到队伍前面去。

这种计算方式被称为**批**（batch）处理：先准备好一些作业，然后由操作员把它们成“批”地运行。那时候，因为成本原因，计算机并不以交互的方式使用：让一个用户坐在计算机前使用它实在太贵了，因为大部分时间它只会闲着，而机房每小时要为此花费数十万美元 [BH00]。

超越库：保护

操作系统不再只是一个提供常用服务的简单库，而是在管理机器方面扮演了更核心的角色。其中一个重要方面，是人们意识到代表操作系统运行的代码是特殊的：它控制着设备，因此应当和普通的应用程序代码区别对待。为什么呢？嗯，想象一下，如果你允许任何应用程序读取磁盘上的任何地方，那隐私就无从谈起了，因为任何程序都可以读取任何文件。因此，把**文件系统**（用来管理你的文件）实现成一个库是没什么道理的。我们需要别的东西。

于是，**系统调用**（system call）的思想被发明了出来，Atlas 计算系统率先采用了它 [K+61, L78]。这个思想不是把操作系统的例程作为库来提供（只需一次过程调用就能访问），而是增加一对特殊的硬件指令和硬件状态，让进入操作系统的过程变成一个更正式、受控的过程。

 © 2008–25, ARPACI-DUSSEAU

系统调用和**过程调用**（procedure call）的关键区别在于：系统调用在把控制权转移（即跳转）到操作系统的同时，还会提升**硬件特权级**（hardware privilege level）。用户应用程序运行在所谓的**用户模式**（user mode）下，这意味着硬件会限制应用程序能做的事情；例如，运行在用户模式下的应用程序通常不能向磁盘发起 I/O 请求、访问任意物理内存页，或者在网络上发送数据包。发起系统调用时（通常是通过一条叫作**陷阱**（trap）的特殊硬件指令），硬件把控制权转移到预先指定的**陷阱处理程序**（trap handler）（由操作系统事先设置好），同时把特权级提升到**内核模式**（kernel mode）。在内核模式下，操作系统可以完全访问系统的硬件，因此可以做诸如发起 I/O 请求或者给程序分配更多内存之类的事情。操作系统处理完请求后，通过一条特殊的**从陷阱返回**（return-from-trap）指令把控制权交还给用户，这条指令会恢复到用户模式，同时把控制权交还给应用程序刚才停下的地方。

多道程序设计时代

操作系统真正腾飞，是在大型机之后的计算时代，也就是**小型机**（minicomputer）时代。像 Digital Equipment 公司的 PDP 系列这样的经典机器，让计算机变得便宜多了；于是，不再是一个大型机构只有一台大型机，机构内部一个较小的群体就很可能拥有自己的计算机。毫不意外，成本下降带来的一个主要影响是开发活动的增加；更多聪明人用上了计算机，于是让计算机系统做出了更多有趣而漂亮的事情。

特别是，出于更好地利用机器资源的愿望，**多道程序设计**（multiprogramming）变得十分普遍。操作系统不再一次只运行一个作业，而是把多个作业装入内存，在它们之间快速切换，从而提高 CPU 利用率。这种切换尤其重要，因为 I/O 设备很慢；让程序在 I/O 处理期间占着 CPU 干等，是在浪费 CPU 时间。为什么不切换到另一个作业、让它运行一会儿呢？

为了支持多道程序设计，以及在存在 I/O 和中断的情况下实现重叠，操作系统在概念上的发展被迫在许多方向上不断创新。**内存保护**（memory protection）之类的问题变得重要起来；我们不希望一个程序能够访问另一个程序的内存。弄清楚如何处理多道程序设计带来的**并发**（concurrency）问题也至关重要；在有中断的情况下确保操作系统行为正确，是一个很大的挑战。我们会在本书后面研究这些问题以及相关的主题。

那个时期的一项主要的实际进展，是 UNIX 操作系统的诞生，这主要归功于贝尔实验室（没错，就是那家电话公司）的 Ken Thompson（和 Dennis Ritchie）。UNIX 从不同的操作系统中吸取了许多好的想法（特别是 Multics [O72]，也有一些来自 TENEX [B+72] 和 Berkeley Time-Sharing System [S68] 这样的系统），但让它们更简单、更易用。很快，这个团队就开始把装有 UNIX 源代码的磁带寄给世界各地的人，其中许多人随后参与进来，自己为系统添砖加瓦；详见**补充**（下一页）[10]。

现代

小型机之后，出现了一种更便宜、更快、面向大众的新型机器：**个人计算机**（personal computer），也就是我们今天所说的 **PC**。在苹果早期机器（例如 Apple II）和 IBM PC 的

[10]我们会用补充框以及其他相关的文本框，来提醒你注意一些不太适合放进正文主线的内容。有时候，我们甚至只是用它们来讲个笑话，因为一路上找点乐子有何不可呢？没错，很多笑话都挺烂的。

 © 2008–25, ARPACI-DUSSEAU

补充：UNIX 的重要性

UNIX 在操作系统历史上的重要性怎么强调都不为过。UNIX 受到更早的一些系统（特别是麻省理工学院著名的 **Multics** 系统）的影响，汇集了许多绝妙的想法，造出了一个既简单又强大的系统。

最初的“贝尔实验室”UNIX 背后，有一条统一的原则：构建一些小而强大的程序，它们可以连接起来，组成更大的工作流。在输入命令的 **shell** 中，提供了**管道**（pipe）这样的原语来支持这种元层次的编程，于是把多个程序串起来完成更大的任务就变得很容易。例如，要找出一个文本文件中含有单词“foo”的行，再数一数这样的行有多少，你只需输入：`grep foo file.txt|wc -l`，就用 `grep` 和 `wc`（word count，统计字数）这两个程序完成了任务。

UNIX 环境对程序员和开发者都很友好，还为新的 **C 编程语言**提供了编译器。让程序员能方便地编写自己的程序，并且分享出去，这使得 UNIX 大受欢迎。作者们把副本免费送给任何索要的人，这大概也起了很大作用，这是**开源软件**（open-source software）的一种早期形式。

同样至关重要的是代码的易得性和可读性。一个用 C 写成的漂亮而小巧的内核，吸引着其他人来摆弄它，加入酷炫的新功能。例如，伯克利一个富有进取心的团队，在 **Bill Joy** 的带领下，做出了一个很棒的发行版（**Berkeley Systems Distribution**，即 **BSD**），它有一些先进的虚拟内存、文件系统和网络子系统。Joy 后来与人共同创立了 **Sun Microsystems**。

遗憾的是，随着各家公司试图主张所有权、从中牟利，UNIX 的传播放慢了一些；这是律师介入之后不幸（但常见）的结果。许多公司都有自己的变体：Sun Microsystems 的 **SunOS**、IBM 的 **AIX**、HP 的 **HPUX**（又名“H-Pucks”），以及 SGI 的 **IRIX**。AT&T/贝尔实验室和这些公司之间的法律纠纷给 UNIX 蒙上了一层阴影，许多人怀疑它能否存活下来，特别是在 Windows 问世并占领了大部分个人计算机市场之后……

带领下，这类新机器很快成为计算领域的主导力量，因为它们的低成本让每张桌子上都能有一台机器，而不再是一个工作组共用一台小型机。

遗憾的是，对操作系统来说，PC 起初是一次大倒退，因为早期的系统忘记了（或者根本不知道）小型机时代学到的教训。例如，像 **DOS**（微软的 **Disk Operating System**）这样的早期操作系统，并不认为内存保护有多重要；于是，一个恶意的（或者只是写得很差的）应用程序就可以在内存中到处乱写。第一代 Mac OS（v9 及更早版本）在作业调度上采用了协作方式；因此，一个不小心陷入无限循环的线程就可能霸占整个系统，迫使系统重启。这一代系统缺失的操作系统功能列出来是一份痛苦而冗长的清单，长到这里没法一一讨论。

幸运的是，在经历了几年的痛苦之后，小型机操作系统的那些老功能开始进入桌面系统。例如，Mac OS X/macOS 以 UNIX 为核心，具备这样一个成熟系统应有的全部功能。Windows 也同样吸收了计算历史上许多好的想法，特别是从 Windows NT 开始，这是微软操作系统技术的一次巨大飞跃。即使是今天的手机，运行的操作系统（例如 Linux）也更像 20 世纪 70 年代小型机上运行的系统，而不像 20 世纪 80 年代 PC 上运行的系统（谢天

 © 2008–25, ARPACI-DUSSEAU

补充：后来有了 Linux

对 UNIX 来说幸运的是，一位名叫 **Linus Torvalds** 的年轻芬兰黑客决定写一个自己的 UNIX。它大量借鉴了原系统背后的原则和思想，但没有借用原来的代码，从而避开了法律问题。他争取到了世界各地许多人的帮助，利用了当时已经存在的成熟 GNU 工具 [G85]，很快，**Linux** 就诞生了（现代开源软件运动也随之诞生）。

进入互联网时代后，大多数公司（例如 Google、Amazon、Facebook 等）都选择运行 Linux，因为它免费，而且可以随时修改以满足自己的需要；事实上，很难想象如果没有这样一个系统，这些新公司还能取得成功。当智能手机成为面向用户的主导平台时，出于许多相同的原因，Linux 在那里也站稳了脚跟（通过 Android）。Steve Jobs 则把他基于 UNIX 的 **NeXTStep** 操作环境带到了苹果，从而让 UNIX 在桌面上流行起来（虽然许多苹果技术的用户大概根本不知道这一点）。因此，UNIX 延续至今，而且比以往任何时候都更重要。如果你信计算之神的话，真该为这个美好的结局感谢它们。

谢地）；看到操作系统发展鼎盛时期诞生的好想法进入了现代世界，真让人高兴。更好的是，这些想法还在不断发展，提供更多功能，让现代系统对用户和应用程序来说变得更好。

2.7 小结

以上就是对操作系统的简介。如今的操作系统让系统相对容易使用，而你今天用到的几乎所有操作系统，都受到了本书将要讨论的那些发展的影响。

遗憾的是，由于时间所限，操作系统中有许多部分本书不会涉及。例如，操作系统中有大量**网络**（networking）代码；这方面的内容，留给你去网络课上学习。同样，**图形**（graphics）设备也特别重要；去上图形学课，拓展你在这方面的知识。最后，有些操作系统书会花大量篇幅讨论**安全**（security）；我们也会讨论，但只是在操作系统必须在运行的程序之间提供保护、并让用户能够保护自己的文件这个意义上，而不会深入讨论你在安全课上可能会学到的那些更深层的安全问题。

不过，我们会讲到许多重要的主题，包括 CPU 和内存虚拟化的基础知识、并发，以及通过设备和文件系统实现的持久性。别担心！虽然要讲的东西很多，但大部分都相当酷，等走到这条路的尽头，你会对计算机系统真正如何工作有全新的认识。现在，开始干活吧！

 © 2008–25, ARPACI-DUSSEAU

参考文献

[BS+09] "Tolerating File-System Mistakes with EnvyFS" by L. Bairavasundaram, S. Sundararaman, A. Arpaci-Dusseau, R. Arpaci-Dusseau. USENIX '09, San Diego, CA, June 2009. *A fun paper about using multiple file systems at once to tolerate a mistake in any one of them.*

[BH00] "The Evolution of Operating Systems" by P. Brinch Hansen. In 'Classic Operating Systems: From Batch Processing to Distributed Systems.' Springer-Verlag, New York, 2000. *This essay provides an intro to a wonderful collection of papers about historically significant systems.*

[B+72] "TENEX, A Paged Time Sharing System for the PDP-10" by D. Bobrow, J. Burchfiel, D. Murphy, R. Tomlinson. CACM, Volume 15, Number 3, March 1972. *TENEX has much of the machinery found in modern operating systems; read more about it to see how much innovation was already in place in the early 1970's.*

[B75] "The Mythical Man-Month" by F. Brooks. Addison-Wesley, 1975. *A classic text on software engineering; well worth the read.*

[BOH10] "Computer Systems: A Programmer's Perspective" by R. Bryant and D. O'Hallaron. Addison-Wesley, 2010. *Another great intro to how computer systems work. Has a little bit of overlap with this book — so if you'd like, you can skip the last few chapters of that book, or simply read them to get a different perspective on some of the same material. After all, one good way to build up your own knowledge is to hear as many other perspectives as possible, and then develop your own opinion and thoughts on the matter. You know, by thinking!*

[G85] "The GNU Manifesto" by R. Stallman. 1985. www.gnu.org/gnu/manifesto.html. *A huge part of Linux's success was no doubt the presence of an excellent compiler,* gcc*, and other relevant pieces of open software, thanks to the GNU effort headed by Stallman. Stallman is a visionary when it comes to open source, and this manifesto lays out his thoughts as to why.*

[K+61] "One-Level Storage System" by T. Kilburn, D.B.G. Edwards, M.J. Lanigan, F.H. Sumner. IRE Transactions on Electronic Computers, April 1962. *The Atlas pioneered much of what you see in modern systems. However, this paper is not the best read. If you were to only read one, you might try the historical perspective below [L78].*

[L78] "The Manchester Mark I and Atlas: A Historical Perspective" by S. H. Lavington. Communications of the ACM, Volume 21:1, January 1978. *A nice piece of history on the early development of computer systems and the pioneering efforts of the Atlas. Of course, one could go back and read the Atlas papers themselves, but this paper provides a great overview and adds some historical perspective.*

[O72] "The Multics System: An Examination of its Structure" by Elliott Organick. MIT Press, 1972. *A great overview of Multics. So many good ideas, and yet it was an over-designed system, shooting for too much, and thus never really worked. A classic example of what Fred Brooks would call the "second-system effect" [B75].*

[PP03] "Introduction to Computing Systems: From Bits and Gates to C and Beyond" by Yale N. Patt, Sanjay J. Patel. McGraw-Hill, 2003. *One of our favorite intro to computing systems books. Starts at transistors and gets you all the way up to C; the early material is particularly great.*

[RT74] "The UNIX Time-Sharing System" by Dennis M. Ritchie, Ken Thompson. CACM, Volume 17: 7, July 1974. *A great summary of* UNIX *written as it was taking over the world of computing, by the people who wrote it.*

[S68] "SDS 940 Time-Sharing System" by Scientific Data Systems. TECHNICAL MANUAL, SDS 90 11168, August 1968. *Yes, a technical manual was the best we could find. But it is fascinating to read these old system documents, and see how much was already in place in the late 1960's. One of the minds behind the Berkeley Time-Sharing System (which eventually became the SDS system) was Butler Lampson, who later won a Turing award for his contributions in systems.*

[SS+10] "Membrane: Operating System Support for Restartable File Systems" by S. Sundararaman, S. Subramanian, A. Rajimwale, A. Arpaci-Dusseau, R. Arpaci-Dusseau, M. Swift. FAST '10, San Jose, CA, February 2010. *The great thing about writing your own class notes: you can advertise your own research. But this paper is actually pretty neat — when a file system hits a bug and crashes, Membrane auto-magically restarts it, all without applications or the rest of the system being affected.*

 © 2008–25, ARPACI-DUSSEAU

作业

本书的大多数章节（最终会是所有章节）末尾都有作业。做这些作业很重要，因为每份作业都能让你（读者）对本章介绍的概念有更多实际体验。

作业有两类。第一类基于**模拟**（simulation）。计算机系统的模拟只是一个简单的程序，它假装完成真实系统所做的一些有趣的工作，然后报告一些输出指标，展示系统的行为。例如，一个硬盘模拟器可以接收一系列请求，模拟它们在具有某种性能特征的硬盘上需要多长时间才能完成，然后报告这些请求的平均延迟。

模拟的妙处在于，它让你能轻松地探索系统的行为，而不必费力去运行一个真实的系统。事实上，它甚至能让你造出现实世界中不可能存在的系统（例如快得难以想象的硬盘），从而看到未来技术可能带来的影响。

当然，模拟也有缺点。就其本质而言，模拟只是对真实系统行为的近似。如果漏掉了现实行为中某个重要的方面，模拟就会报告出错误的结果。因此，对模拟的结果总要抱有几分怀疑。归根结底，真正重要的是系统在现实世界中的表现。

第二类作业需要和**真实世界的代码**（real-world code）打交道。其中有些作业侧重测量，另一些只需要做一点小规模的开发和实验。这两种作业都只是对一个更大世界的小小探索，而这个世界正是你应该走进去的：如何在基于 UNIX 的系统上用 C 编写系统代码。事实上，要推动你朝这个方向发展，就需要比这些作业规模更大的项目；因此，除了做作业，我们强烈建议你做一些项目，来巩固你的系统技能。一些项目见这个页面（https://github.com/remzi-arpacidusseau/ostep-projects）。

要做这些作业，你很可能需要一台基于 UNIX 的机器，运行 Linux、macOS 或类似的系统。它还应该装有 C 编译器（例如 `gcc`）和 Python。你还应该会用某种真正的代码编辑器来编辑代码。

 © 2008–25, ARPACI-DUSSEAU